

A DPP Scheme with Zero-Knowledge Traceability

January 20, 2026

Contents

1	System Model and Goals	2
2	Lifecycle State Machine	2
2.1	States and Notes	2
2.2	Policy and Allowed Transitions	2
2.3	Event-Typed State Evolution	3
2.4	Counter Update Rules	3
2.5	Invariants and Enforcement Split	4
3	Zero-Knowledge Proof Layer	4
3.1	Proof Types	5
3.2	Who Proves and Who Verifies	5
3.3	What ZK Is Used For	5
3.4	Which Events Use ZK Under Each Mode	5
4	System Architecture	6
4.1	Design Principle: On-chain Consensus, Off-chain Computation	6
4.2	On-chain Modules	6
4.3	Off-chain Services	7
4.4	Interfaces	7
5	Per-event Workflow	7
5.1	RegisterRaw (certificate-gated admission)	7
5.2	Produce ($m \rightarrow n$ production/sorting with waste partition)	8
5.3	Transfer (ownership transfer)	8
5.4	Recycle (recyclable waste to raw with $rc+=1$)	8
5.5	Dispose and Consume (termination)	9
6	System Fields (Compact)	9
7	ZK Circuits and IVC-style Passport Proof	9
7.1	Common Interfaces and Gadgets	9
7.2	Produce Circuit C_{Prod}	9
7.3	Recycle Circuit C_{Rec}	10
7.4	Transfer Circuit C_{Tx} (privacy-on only)	10
7.5	Recursive Passport Proof via IVC	11
8	Embedding an LLM-based Proof Generator for Practical Deployment	12
8.1	Motivation	12
8.2	System Components	12
8.3	End-to-end Workflow for Non-expert Users	13
8.4	Correctness, Safety, and Determinism	14
8.5	Template Management and Updates	14
8.6	Cost and Deployment Considerations	14

1 System Model and Goals

External certification is out of scope. We assume real-world authorities (e.g., government agencies, food regulators, labs) issue certificates/reports to participants and/or batches. The issuance/revocation lifecycle is *external* to our DPP system. Our DPP only *references* such credentials, either publicly or privately via ZK.

Goals. Our DPP targets:

- **Full-lifecycle traceability:** every accepted output item can be traced back to valid prior events without revealing business secrets.
- **Anti-fraud and anti-reuse:** certified registrations cannot be replayed; items cannot be consumed/spent twice; recyclable waste cannot be inflated via loops.
- **Confidentiality:** hide sensitive attributes (BOM/recipe, suppliers, logistics, costs, detailed measurements) while proving policy compliance and conservation laws.

2 Lifecycle State Machine

2.1 States and Notes

Lifecycle states. We model each batch/item as carrying a discrete lifecycle state

$$st \in \Sigma := \{\text{raw}, \text{prod}, \text{waste}^{\text{in}}, \text{waste}^{\text{out}}, \text{spent}\}.$$

Here *raw* denotes admissible raw materials, *prod* denotes in-market products, *wasteⁱⁿ* denotes *recyclable* waste eligible for the Recycle event, *waste^{out}* denotes *non-recyclable* waste eligible only for disposal/termination, and *spent* denotes termination.

Counters (circular economy semantics). Each note additionally carries counters

$$(\text{pid}, \text{c}, \text{rc}),$$

where *pid* identifies a production line/epoch, $\text{c} \in \{0, 1, \dots, K\}$ is a stage counter within that line, and $\text{rc} \in \mathbb{N}$ is a recycle-count that records how many recycling cycles the material has undergone across lines. The recycle-count *rc* is *monotone* and can increase only via the Recycle event.

Private notes and public commitments. A private note is

$$\text{note} = (st, \text{pid}, \text{c}, \text{rc}, \text{owner}, \text{qty}, \text{attrs_com}, \text{policy}, \text{cred_com}, \text{link}),$$

and the ledger stores only its commitment $cm = \text{Com}(\text{note}; r)$. The lifecycle state machine governs the admissible evolution of $(st, \text{pid}, \text{c}, \text{rc})$ across events.

2.2 Policy and Allowed Transitions

Policy. A policy identifier *policy* fixes: (i) an allowed-transition relation $\text{Allowed}_{\text{policy}}$; (ii) deterministic update rules for $(\text{pid}, \text{c}, \text{rc})$ per event type; (iii) a traceability-link rule binding outputs to consumed inputs; and (iv) optional compliance predicates for public claims *claim* (e.g., emissions thresholds, recycled-content grades). All lifecycle validity statements are interpreted relative to *policy*.

Allowed-transition relation. We define

$$\text{Allowed}_{\text{policy}} : \Sigma \times \Sigma \rightarrow \{0, 1\}$$

where $\text{Allowed}_{\text{policy}}(s, t) = 1$ iff a note in state *s* is allowed to transition to state *t* under policy *policy* by some admitted event.

Baseline edges (including pre-production waste). To capture spoilage, contamination, transport damage, or inspection failures *before* production, we allow raw materials to become waste directly. Concretely, we fix:

$$\begin{aligned} \text{Allowed}(\text{raw}, \text{prod}) &= 1, & \text{Allowed}(\text{raw}, \text{waste}^{\text{in}}) &= 1, & \text{Allowed}(\text{raw}, \text{waste}^{\text{out}}) &= 1, \\ \text{Allowed}(\text{prod}, \text{waste}^{\text{in}}) &= 1, & \text{Allowed}(\text{prod}, \text{waste}^{\text{out}}) &= 1, \\ \text{Allowed}(\text{waste}^{\text{in}}, \text{raw}) &= 1 & (\text{realized by the Recycle event}), \\ \text{Allowed}(\text{waste}^{\text{out}}, \text{spent}) &= 1, & \text{Allowed}(\text{prod}, \text{spent}) &= 1. \end{aligned}$$

For transportation/trade (ownership transfer), we additionally allow self-loops:

$$\text{Allowed}(st, st) = 1 \text{ for Transfer.}$$

All other state transitions are disallowed.

Why an explicit transition relation is necessary. Nullifiers alone prevent double-spending but do not prevent *semantic forgery*: without $\text{Allowed}_{\text{policy}}$, a party could spend a valid note and arbitrarily label the output state (e.g., treat $\text{waste}^{\text{out}}$ as recyclable), breaking circular-economy semantics and enabling recycled-content inflation. Our protocol therefore enforces $\text{Allowed}_{\text{policy}}$ as a first-class constraint.

2.3 Event-Typed State Evolution

Event alphabet. Lifecycle evolution is driven by typed events

$$\mathcal{E} = \{\text{RegisterRaw}, \text{Produce}, \text{Transfer}, \text{Recycle}, \text{Dispose}, \text{Consume}\},$$

each consuming a multiset of input notes and producing a multiset of output notes (or terminating notes).

Typing rules (consumed/produced states). We enforce the following typing rules (either inside ZK circuits when privacy is enabled, or as public checks otherwise):

- **RegisterRaw**: creates a single note with $st = \text{raw}$ (certificate-gated admission).
- **Produce** ($m \rightarrow n$): consumes notes with $st \in \{\text{raw}, \text{prod}\}$ and produces notes with $st' \in \{\text{prod}, \text{waste}^{\text{in}}, \text{waste}^{\text{out}}\}$. (This covers pre-production waste via $\text{raw} \rightarrow \text{waste}^{\text{in}}/\text{waste}^{\text{out}}$ and post-production byproducts via $\text{prod} \rightarrow \text{waste}^{\text{in}}/\text{waste}^{\text{out}}$.)
- **Transfer**: consumes one note with state st and produces one note with the same st (ownership changes only).
- **Recycle**: consumes one note with $st = \text{waste}^{\text{in}}$ and produces one note with $st' = \text{raw}$.
- **Dispose**: consumes one note with $st = \text{waste}^{\text{out}}$ and terminates it (state becomes spent).
- **Consume**: consumes one note with $st = \text{prod}$ and terminates it (state becomes spent).

These typing rules imply: (i) $\text{waste}^{\text{out}}$ cannot be recycled; (ii) only waste^{in} may enter recycling; and (iii) termination is explicit and verifiable.

2.4 Counter Update Rules

Recycle-count update (the only source of rc increments). The recycle-count rule is:

$$rc' = \begin{cases} rc + 1, & \text{if the event is Recycle;} \\ rc, & \text{otherwise.} \end{cases}$$

Thus, each successful recycling cycle is explicitly recorded in the note metadata.

Pipeline counter and line identifier. The pair (pid, c) encodes *where* and *when* a note is created within production:

- In Produce, each output sets $c' = \max_i c_i + 1$ (or a policy-defined increment rule) and typically preserves pid (same line/epoch).
- In Recycle, the output raw resets $c' = 0$ and sets pid' according to policy: either $pid' = pid$ (re-enters the same line) or pid' equals a designated new line identifier (re-enters a new line).
- In Transfer, (pid, c, rc) are unchanged.
- In Dispose/Consume, the note terminates; counters are irrelevant thereafter.

These rules prevent ambiguous “teleportation” across lines and make cross-cycle history explicit.

2.5 Invariants and Enforcement Split

Invariant I (no illegal recycling, no hidden rc increments). A note can increase rc only via Recycle, and Recycle is applicable only to $waste^{in}$. Hence, $waste^{out}$ can never become raw, and recycled-content accumulation cannot be claimed without an explicit recycle event.

Invariant II (anti-loop via evidence freshness). To prevent infinite inflation using repeated recycling evidence, each Recycle event carries a public evidence-id eid_{rec} , which must be fresh w.r.t. $EidSet^{rec}$ at acceptance time. This freshness check is ledger-enforced.

Invariant III (certificate anti-reuse for admission). Each RegisterRaw event carries a certificate evidence-id eid_{cert} , which must be fresh w.r.t. $EidSet^{cert}$ at acceptance time, preventing a single certificate from registering many fake batches.

Invariant IV (termination is verifiable). Dispose and Consume terminate notes by revealing a nullifier nf (ledger-checked for freshness) and recording the event in EventLog.

Enforcement split: ZK vs ledger-only. The state-machine constraints (typing rules, $Allowed_{policy}$, and counter updates) are enforced: (i) inside per-event ZK circuits whenever the deployment hides suppliers/trading relationships or confidential process witnesses; or (ii) as public transaction checks when all relevant fields are revealed. In all cases, global freshness—nullifiers in NullSet and evidence IDs in $EidSet^{rec}/EidSet^{cert}$ —is enforced by the ledger.

3 Zero-Knowledge Proof Layer

Design principle. Our DPP protocol separates (i) *ledger acceptance* of lifecycle events from (ii) *presentation* of lifecycle history. The former requires per-event validity checks under privacy, while the latter (optionally) compresses long histories for efficient third-party verification.

Two deployment modes. Our system supports two interoperable deployment modes, selected by the policy identifier policy:

- **Privacy-on (default):** hide suppliers and trading relationships (who transacts with whom and which item is transferred), while preserving public verifiability of lifecycle correctness.
- **Privacy-off (optional):** do not hide suppliers/trading relationships; events are validated publicly with signatures and explicit references, reducing proof overhead.

Crucially, both modes share the same lifecycle state machine, freshness checks (nullifiers/evidence IDs), and traceability semantics; they differ only in which event relations are proven in zero knowledge.

3.1 Proof Types

Type I: Per-event SNARK (ledger-acceptance proof). For an event type `etype` that requires privacy under policy, the event generator produces a succinct proof

$$\pi_{\text{etype}} \leftarrow \text{Prove}(pk_{\text{etype}}, x_{\text{etype}}; w_{\text{etype}}), \quad \text{Verify}(vk_{\text{etype}}, x_{\text{etype}}, \pi_{\text{etype}}) = 1.$$

The public transcript x_{etype} includes ledger roots, nullifiers, output commitments, evidence IDs, and optional public claims. The witness w_{etype} contains all sensitive information (owners, suppliers, recipes/BOM, process measurements, private quantities, Merkle paths, randomness).

Type II (optional): Recursive/IVC passport proof (presentation). Optionally, a recursive SNARK/IVC aggregates a sequence of accepted Type-I proofs into one “passport proof” for efficient third-party verification. This layer is used only for *presentation* and does not replace ledger freshness enforcement.

3.2 Who Proves and Who Verifies

Prover (event generator). The prover is the actor executing the event: manufacturers prove **Produce**, recyclers prove **Recycle**, and transacting parties prove **Transfer only in privacy-on mode**. For non-ZK events, the actor submits a publicly verifiable transaction (e.g., signatures and explicit references).

Ledger verifier (acceptance authority). The ledger validates each event as follows:

- **If the event is ZK-enabled under policy:** verify the SNARK proof π_{etype} .
- **Always (all modes):** enforce global freshness: (i) reject reused nullifiers nf via **NullSet**; (ii) reject reused certificate evidence IDs eid_{cert} via $\text{EidSet}^{\text{cert}}$; (iii) reject reused recycling evidence IDs eid_{rec} via $\text{EidSet}^{\text{rec}}$.
- **Upon acceptance:** append output commitments to **NoteTree** (if any), insert nf and evidence IDs into the corresponding sets, and append the transcript to **EventLog**.

Third-party verifier. Regulators/auditors/consumers can verify: (i) public transactions (privacy-off) and public claims; and/or (ii) Type-I SNARK proofs for privacy-on events; and optionally (iii) a single aggregated passport proof.

3.3 What ZK Is Used For

ZK provides private event validity. When enabled, ZK is used to prove, without revealing business-sensitive witnesses:

1. **Authorized spending:** each consumed note exists in **NoteTree** (Merkle inclusion) and is controlled by the prover (nullifier correctness).
2. **Lifecycle semantics:** state transitions satisfy $\text{Allowed}_{\text{policy}}$ and counter update rules for $(\text{pid}, \text{c}, \text{rc})$.
3. **Conservation constraints:** exact quantity conservation in **Produce** and bounded yield in **Recycle**.
4. **Traceability binding:** outputs carry link derived from consumed input commitments.
5. **Optional compliance claims:** e.g., $\text{emissions} \leq T$ or $\text{recycled ratio} \geq x$, without revealing underlying measurements.

3.4 Which Events Use ZK Under Each Mode

Privacy-on (default): hide suppliers/trading relationships.

- **Transfer: ZK-enabled.** This hides which item is transferred and hides the counterparties (supplier/customer relationships), while still proving authorized spending and correct re-commitment.
- **Produce: ZK-enabled.** This hides recipes/BOM, measurements, and (optionally) quantities while proving policy-compliant evolution and conservation.

- **Recycle: ZK-enabled.** This hides recycling witnesses while proving $\text{waste}^{\text{in}} \rightarrow \text{raw}$ with $\text{rc}+=1$, bounded yield, and evidence binding.
- **RegisterRaw, Dispose, Consume: public** by default. Certificate-gated admission and termination do not require ZK if no sensitive witness must be hidden.

Privacy-off (optional): suppliers/trades are public.

- **Transfer: public.** Parties reveal the spent reference and prove authorization via standard signatures; the ledger still enforces nf freshness.
- **Produce, Recycle: remain ZK-enabled** if process details are confidential; otherwise they can also be public if the deployment reveals all witnesses.
- **RegisterRaw, Dispose, Consume: public.**

Optional certificate privacy. If certificate contents or issuer–batch linkage is considered sensitive, RegisterRaw can be switched to a ZK variant that proves possession/validity of a certificate while revealing only eid_{cert} (still checked for one-time use by $\text{EidSet}^{\text{cert}}$).

4 System Architecture

4.1 Design Principle: On-chain Consensus, Off-chain Computation

Our protocol follows a standard separation of concerns: the blockchain enforces *global consistency* (immutability, ordering, and freshness), while off-chain services perform *heavy computation* (proof generation) and handle confidential data. This division is necessary because (i) SNARK proving is computationally intensive, and (ii) business-sensitive information (e.g., suppliers, recipes, detailed measurements) must not be posted on-chain.

On-chain responsibilities. The blockchain (via smart contracts or native ledger logic) is responsible for: (i) verifying event validity (SNARK verification for ZK-enabled events or signature checks for public events); (ii) enforcing *freshness* of spent objects and evidence identifiers; (iii) maintaining public state commitments (e.g., Merkle roots) that anchor traceability; and (iv) recording an append-only public transcript of accepted events.

Off-chain responsibilities. Off-chain servers (operated by supply-chain participants or delegated service providers) are responsible for: (i) generating SNARK proofs for witness-dependent events; (ii) constructing note commitments; (iii) parsing external certificates and recycling evidence; and (iv) storing sensitive operational data. Off-chain services submit the resulting public transcript (and proof, when required) to the chain for acceptance.

4.2 On-chain Modules

We implement the on-chain logic as a dispatcher contract with three logical modules.

(i) Commitment module (state anchor). The contract maintains a public commitment structure NoteTree over note commitments cm . Accepted events may append one or more new commitments and update the public root rt_{cm} .

(ii) Freshness module (anti-replay). The contract maintains three public sets (or Merkle accumulators):

- a nullifier set NullSet to prevent double-spending of consumed notes;
- a certificate evidence-id set $\text{EidSet}^{\text{cert}}$ to prevent reusing one admission certificate to register multiple fake batches; and
- a recycling evidence-id set $\text{EidSet}^{\text{rec}}$ to prevent infinite loop inflation by replaying the same recycling evidence.

These checks are enforced at acceptance time for every relevant event.

(iii) Verifier/dispatcher module. The dispatcher exposes a single entry point `SubmitEvent` that: (a) verifies a SNARK proof if the event type is ZK-enabled under policy `policy`; (b) otherwise verifies standard signatures and public references; (c) enforces freshness via `NullSet` and evidence-id sets; (d) updates `NoteTree` by appending newly created commitments; and (e) appends the public transcript to an append-only event log `EventLog`.

4.3 Off-chain Services

We describe a canonical off-chain stack. Implementations may co-locate these functions or split them across services.

(i) Prover service. For ZK-enabled events (notably `Produce` and `Recycle`, and optionally `Transfer` in privacy-on mode), the prover service constructs witnesses, evaluates the event circuit, and generates the SNARK proof π .

(ii) Confidential data vault. A data vault stores sensitive operational records (supplier identity, BOM/recipes, fine-grained measurements, ERP logs). Only commitments (`attrs_com`) and optionally coarse claims `claim` are exposed.

(iii) Evidence adapter. An adapter parses external admission certificates and recycling evidence, computes evidence identifiers $\text{eid}_{\text{cert}} = H(\text{cred})$ and $\text{eid}_{\text{rec}} = H(\text{rec.evidence})$, and (optionally) prepares witness material for ZK circuits.

(iv) Transaction builder/relayer. A relayer bundles (etype, x, π) (or $(\text{etype}, \text{payload})$ for public events) into a blockchain transaction and submits it. It can also batch outputs (e.g., multiple `cm` from one production step) to reduce cost.

4.4 Interfaces

On-chain interface. We assume the following minimal contract interface:

- `SubmitEvent(etype, x,  $\pi$ )` for ZK-enabled events;
- `SubmitPublicEvent(etype, payload)` for public events;
- `GetRoots()` returning current rt_{cm} (and optional accumulator roots for `NullSet/`);
- (optional) `GetEvent(id)` or range queries to retrieve event transcripts for auditing.

Off-chain interface. Off-chain services expose API endpoints such as: `ProveProduce`, `ProveRecycle`, and (optional) `ProveTransfer`, each returning (x, π) ready for on-chain submission.

5 Per-event Workflow

We now describe each event as a protocol between an event generator (prover), the ledger (verifier/acceptor), and optional third-party auditors.

5.1 RegisterRaw (certificate-gated admission)

Goal. Admit a new raw-material batch, anchored by a public commitment cm_{out} , while preventing certificate replay via eid_{cert} .

Off-chain (event generator). Given an external certificate `cred` (issued by an external authority, out of scope), the generator computes $\text{eid}_{\text{cert}} = H(\text{cred})$, constructs a note with $\text{st} = \text{raw}$ and counters $(\text{pid}, 0, \text{rc})$, and computes $\text{cm}_{\text{out}} = \text{Com}(\text{note}; r)$. Optionally, if certificate contents must remain private, the generator produces a SNARK proof that it possesses a valid certificate bound to this batch while revealing only eid_{cert} .

On-chain (ledger acceptance). The contract checks $\text{eid}_{\text{cert}} \notin \text{EidSet}^{\text{cert}}$ and inserts it upon acceptance. It appends cm_{out} to **NoteTree** and records the event transcript in **EventLog**. (Any certificate verification can be performed publicly or treated as an external assumption, depending on deployment.)

5.2 Produce ($m \rightarrow n$ production/sorting with waste partition)

Goal. Consume m input notes and create n output notes in states $\{\text{prod}, \text{waste}^{\text{in}}, \text{waste}^{\text{out}}\}$, enforcing policy-compliant state evolution, counter updates, and exact conservation, while keeping process witnesses private.

Off-chain (prover). The producer gathers the private input notes and membership paths in **NoteTree**. It chooses output notes (including waste partition), sets counters per policy, and computes output commitments $\{cm'_j\}$. It then generates a SNARK proof

$$\pi_{\text{Prod}} \leftarrow \text{Prove}(pk_{\text{Prod}}, x_{\text{Prod}}; w_{\text{Prod}}),$$

where x_{Prod} includes (**policy**, rt_{cm} , rt_{nf} , $\{nf_i\}$, $\{cm'_j\}$, **claim**) and the witness enforces: (i) inclusion of each input commitment in **NoteTree**; (ii) nullifier correctness and ownership authorization; (iii) **Allowed**_{policy} and event typing ($\text{raw} \rightarrow \text{prod}/\text{waste}^{\text{in}}/\text{waste}^{\text{out}}$); (iv) counter updates for (**pid**, **c**, **rc**); (v) exact quantity conservation across outputs (including pre-production $\text{raw} \rightarrow \text{waste}^*$); (vi) binding of outputs via link to consumed inputs; and (vii) optional compliance claim predicates **claim**.

On-chain (ledger acceptance). The contract verifies π_{Prod} . It checks freshness of all consumed nullifiers $\{nf_i\}$ against **NullSet** and inserts them. It appends output commitments $\{cm'_j\}$ to **NoteTree** and records the transcript in **EventLog**.

5.3 Transfer (ownership transfer)

Goal. Transfer ownership of a note without changing lifecycle state or counters.

Privacy-on mode (hide suppliers/trading relationships). The sender proves authorized spending of an input note and correct re-commitment to a new owner without revealing the spent reference. Off-chain, the prover generates π_{Tx} with public transcript $x_{\text{Tx}} = (\text{rt}_{cm}, \text{rt}_{nf}, nf_{in}, cm_{out})$. On-chain, the contract verifies π_{Tx} , checks nf_{in} freshness, inserts it, appends cm_{out} , and logs the event.

Privacy-off mode (public trading). If relationships are public, the sender submits a public transaction that references the spent item and carries a standard signature. On-chain, the contract verifies signatures, checks nf_{in} freshness, inserts it, appends cm_{out} , and logs the event. No ZK proof is required.

5.4 Recycle (recyclable waste to raw with $\text{rc} += 1$)

Goal. Convert a recyclable waste note into a new raw note, incrementing recycle-count **rc** and preventing loop inflation.

Off-chain (prover). The recycler gathers the input waste note (which must satisfy $st = \text{waste}^{\text{in}}$) and its membership path, parses recycling evidence **rec_evidence**, and computes $\text{eid}_{\text{rec}} = H(\text{rec_evidence})$. It constructs an output raw note with $\text{rc}' = \text{rc} + 1$ and $\text{c}' = 0$ (and **pid'** per policy), computes cm_{out} , and generates a SNARK proof π_{Rec} with transcript $x_{\text{Rec}} = (\text{policy}, \text{rt}_{cm}, \text{rt}_{nf}, nf_{in}, cm_{out}, \text{eid}_{\text{rec}}, \text{claim})$. The witness enforces: (i) input inclusion in **NoteTree** and authorized spending; (ii) typing and **Allowed**_{policy} for $\text{waste}^{\text{in}} \rightarrow \text{raw}$; (iii) counter updates $\text{rc}' = \text{rc} + 1$ and $\text{c}' = 0$ and policy-consistent **pid'**; (iv) bounded yield/range constraints to prevent minting raw material from nothing; and (v) binding of the evidence to the consumed input (preventing “borrowed” receipts).

On-chain (ledger acceptance). The contract verifies π_{Rec} . It checks nf_{in} freshness via **NullSet** and eid_{rec} freshness via $\text{EidSet}^{\text{rec}}$, and inserts both. It appends cm_{out} to **NoteTree** and records the transcript in **EventLog**.

5.5 Dispose and Consume (termination)

Goal. Terminate non-recyclable waste (Dispose) or products (Consume) in a publicly verifiable manner.

Off-chain (event generator). The actor constructs a public termination transaction, computes the nullifier nf_{in} , and signs the transaction. Optionally, it may attach a receipt hash for audit.

On-chain (ledger acceptance). The contract verifies signatures, checks $nf_{in} \notin \text{NullSet}$, inserts it, and records the termination event in `EventLog`. No output commitments are created.

6 System Fields (Compact)

Compact field dictionary is Table 1.

7 ZK Circuits and IVC-style Passport Proof

7.1 Common Interfaces and Gadgets

Note commitment and nullifier. A private note is

$$\text{note} = (st, \text{pid}, c, rc, \text{owner}, \text{qty}, \text{attrs_com}, \text{policy}, \text{cred_com}, \text{link}),$$

with commitment

$$cm = \text{Com}(\text{note}; r).$$

A spender derives an owner-id and a nullifier as

$$\text{owner} = \text{KeyDer}(sk), \quad nf = \text{PRF}(sk, cm).$$

Merkle membership gadget. Let `NoteTree` be a Merkle tree over commitments with public root rt_{cm} . We use an abstract gadget $\text{MTMem}(\text{rt}_{cm}, cm, \pi^{\text{mt}}) = 1$ to assert membership, without expanding its internal constraints.

Ledger-only freshness. Freshness checks are *not* in circuits: the ledger enforces $nf \notin \text{NullSet}$, $\text{eid}_{\text{cert}} \notin \text{EidSet}^{\text{cert}}$, and $\text{eid}_{\text{rec}} \notin \text{EidSet}^{\text{rec}}$.

Policy-selected claims (optional). We represent an optional public claim as $\text{claim} = (\text{ct}, \text{cv})$, and use a policy-selected gadget $\text{ClaimCheck}_{\text{policy}}(\text{aux}) = \text{claim}$.

7.2 Produce Circuit C_{Prod}

Public input.

$$x_{\text{Prod}} = (\text{policy}, \text{rt}_{cm}, \{nf_i\}_{i=1}^m, \{cm'_j\}_{j=1}^n, \text{claim}).$$

Private witness.

$$w_{\text{Prod}} = \left(\{\text{note}_i, r_i, sk_i, \pi_i^{\text{mt}}\}_{i=1}^m, \{\text{note}'_j, r'_j\}_{j=1}^n, \text{aux} \right).$$

Constraints. $C_{\text{Prod}}(x_{\text{Prod}}; w_{\text{Prod}}) = 1$ iff all constraints below hold:

- P1. Policy binding.** $\text{policy}_i = \text{policy}$ for all inputs and $\text{policy}'_j = \text{policy}$ for all outputs.
- P2. Commitment correctness (inputs/outputs).** For all i , $cm_i = \text{Com}(\text{note}_i; r_i)$; for all j , $cm'_j = \text{Com}(\text{note}'_j; r'_j)$ and equals the corresponding public cm'_j .
- P3. Input membership.** For all i , $\text{MTMem}(\text{rt}_{cm}, cm_i, \pi_i^{\text{mt}}) = 1$.
- P4. Authorized spending and nullifiers.** For all i , $\text{owner}_i = \text{KeyDer}(sk_i)$ and $nf_i = \text{PRF}(sk_i, cm_i)$, and each nf_i equals the corresponding public nf_i .
- P5. Typing (state constraints).** For all i , $st_i \in \{\text{raw}, \text{prod}\}$; for all j , $st'_j \in \{\text{prod}, \text{waste}^{\text{in}}, \text{waste}^{\text{out}}\}$.

- P6. Counter alignment and update (single-step model).** All inputs share one line/stage: $\text{pid}_i = \text{pid}_{\text{in}}, c_i = c_{\text{in}}, rc_i = rc_{\text{in}}$ for all i ; all outputs satisfy $\text{pid}'_j = \text{pid}_{\text{in}}, rc'_j = rc_{\text{in}}, c'_j = c_{\text{in}} + 1$ for all j .
- P7. Exact conservation.** $\sum_{i=1}^m \text{qty}_i = \sum_{j=1}^n \text{qty}'_j$ (with range checks on all quantities).
- P8. Traceability binding.** Define $\text{link}^* = H(\text{tag}_{\text{link}} \parallel \text{policy} \parallel cm_1 \parallel \dots \parallel cm_m)$, and enforce $\text{link}'_j = \text{link}^*$ for all j .
- P9. Optional claim.** If $\text{claim} \neq \perp$, enforce $\text{ClaimCheck}_{\text{policy}}(\text{aux}) = \text{claim}$.

Explanation. P1–P4 prove that the prover controls the consumed notes (membership + ownership) while revealing only nullifiers. P5 enforces allowed output categories, including pre-production waste via $\text{raw} \rightarrow \text{waste}^{\text{in}}/\text{waste}^{\text{out}}$. P6 avoids an expensive $\max(c)$ gadget by modeling one production step as consuming inputs from a single stage. P7 enforces non-inflation via exact conservation. P8 binds outputs to the consumed inputs for provenance and passport aggregation.

7.3 Recycle Circuit C_{Rec}

Public input.

$$x_{\text{Rec}} = (\text{policy}, \text{rt}_{cm}, nf_{\text{in}}, cm_{\text{out}}, \text{eid}_{\text{rec}}, \text{claim}).$$

Private witness.

$$w_{\text{Rec}} = (\text{note}_{\text{in}}, r_{\text{in}}, sk, \pi^{\text{mt}}, \text{note}_{\text{out}}, r_{\text{out}}, \text{rec_evidence}, \text{aux}).$$

Constraints. $C_{\text{Rec}}(x_{\text{Rec}}; w_{\text{Rec}}) = 1$ iff:

- R1. Policy binding.** $\text{policy}_{\text{in}} = \text{policy}_{\text{out}} = \text{policy}$.
- R2. Commitment correctness and membership.** $cm_{\text{in}} = \text{Com}(\text{note}_{\text{in}}; r_{\text{in}})$ and $\text{MTMem}(\text{rt}_{cm}, cm_{\text{in}}, \pi^{\text{mt}}) = 1$; $cm_{\text{out}} = \text{Com}(\text{note}_{\text{out}}; r_{\text{out}})$ and equals public cm_{out} .
- R3. Authorized spending and nullifier.** $\text{owner}_{\text{in}} = \text{KeyDer}(sk)$ and $nf_{\text{in}} = \text{PRF}(sk, cm_{\text{in}})$, and equals public nf_{in} .
- R4. Typing.** $st_{\text{in}} = \text{waste}^{\text{in}}$ and $st_{\text{out}} = \text{raw}$.
- R5. Counter updates.** $rc_{\text{out}} = rc_{\text{in}} + 1$, $c_{\text{out}} = 0$, and pid_{out} follows policy (same-line or policy-defined new line).
- R6. Bounded yield (no minting).** Enforce $\text{qty}_{\text{out}} \leq \alpha_{\text{policy}} \cdot \text{qty}_{\text{in}}$ (implemented with fixed-point bounds and range checks).
- R7. Evidence-id binding.** $\text{eid}_{\text{rec}} = H(\text{tag}_{\text{eidrec}} \parallel \text{Enc}(\text{rec_evidence}))$ equals public eid_{rec} , and is bound to the consumed input (e.g., absorbed into link_{out}):

$$\text{link}_{\text{out}} = H(\text{tag}_{\text{link}} \parallel \text{policy} \parallel cm_{\text{in}} \parallel \text{eid}_{\text{rec}}).$$

- R8. Optional claim.** If $\text{claim} \neq \perp$, enforce $\text{ClaimCheck}_{\text{policy}}(\text{aux}) = \text{claim}$.

Explanation. R2–R3 prove the recycler controls the consumed waste note. R4–R5 enforce that only recyclable waste can create new raw material and that each recycling increments rc . R6 prevents inflation by bounding yield, while the ledger additionally prevents replay by enforcing freshness of eid_{rec} . R7 binds the recycling evidence to the consumed input and the produced output.

7.4 Transfer Circuit C_{Tx} (privacy-on only)

Public input.

$$x_{\text{Tx}} = (\text{policy}, \text{rt}_{cm}, nf_{\text{in}}, cm_{\text{out}}).$$

Private witness.

$$w_{\text{Tx}} = (\text{note}_{\text{in}}, r_{\text{in}}, sk, \pi^{\text{mt}}, \text{owner}_{\text{out}}, \text{note}_{\text{out}}, r_{\text{out}}).$$

Constraints. $C_{Tx}(x_{Tx}; w_{Tx}) = 1$ iff:

T1. Input commitment and membership. $cm_{in} = \text{Com}(\text{note}_{in}; r_{in})$ and $\text{MTMem}(\text{rt}_{cm}, cm_{in}, \pi^{\text{mt}}) = 1$.

T2. Authorized spending and nullifier. $\text{owner}_{in} = \text{KeyDer}(sk)$ and $nf_{in} = \text{PRF}(sk, cm_{in})$, equals public nf_{in} .

T3. State/counter preservation. All fields except owner (and randomness) are preserved:

$$(st, pid, c, rc, qty, \text{attrs_com}, \text{policy}, \text{cred_com}, \text{link})_{out} = (\cdot)_{in}.$$

T4. Output commitment. $cm_{out} = \text{Com}(\text{note}_{out}; r_{out})$ equals public cm_{out} , and owner_{out} is the new spending authority.

Explanation. This circuit hides the spent reference and counterparties while proving that ownership changes only through authorized spending. In privacy-off deployments, **Transfer** is a public signed transaction and no ZK circuit is used.

7.5 Recursive Passport Proof via IVC

Problem with offline aggregation. If a passport proof were generated *offline* by a single party after the fact, it would require access to all per-step private witnesses (recipes, measurements, receipts), which is impractical. Therefore, our passport is generated *incrementally*: each event generator extends the passport once using only its local witness.

Passport artifact carried with the lineage head. Along with the (committed) lineage head, we maintain a recursive passport artifact:

$$\mathcal{P}_k := (\Pi_k, D_k),$$

where Π_k is an IVC/recursive proof for k steps and D_k is a rolling digest that binds the exact on-chain event sequence. This artifact is transferred together with ownership (as off-chain payload) and can be hashed into the output note commitment.

On-chain event-log anchoring. Let the chain publish an event-log digest rt_{elog} (e.g., Merkle root) over leaves

$$\ell = H(\text{tag}_{\text{elog}} \parallel \text{Enc}(\text{etype}) \parallel \text{Enc}(x)).$$

For each event at step k , the event generator obtains an inclusion proof π_k^{elog} that $\ell_k \in \text{rt}_{\text{elog}}$.

Passport state. We define a compact recursion state

$$S_k = (cm_k, st_k, pid_k, c_k, rc_k, D_k),$$

where cm_k is the current lineage head commitment and D_k binds all accepted events up to step k .

Incremental step proving interface. Each event generator at step k runs the IVC prover to extend the passport:

$$(\Pi_k, S_k) \leftarrow \text{IVC.Prove}(\Pi_{k-1}, S_{k-1}; \text{rt}_{\text{elog}}, \text{etype}_k, x_k, \pi_k^{\text{elog}}, w_k),$$

where w_k is the *local* private witness for the current event only. No party ever needs to hold witnesses from other steps.

Step relation (constraints). The step circuit (relation) verified by Π_k enforces:

S1. On-chain anchoring. Compute $\ell_k = H(\text{tag}_{\text{elog}} \parallel \text{Enc}(\text{etype}_k) \parallel \text{Enc}(x_k))$ and check $\text{MTMem}(\text{rt}_{\text{elog}}, \ell_k, \pi_k^{\text{elog}}) = 1$.

S2. Continuity of the lineage head. Parse from x_k a public input-head commitment $cm_{in}^{(k)}$ and a set of public output commitments $\{cm_{out}^{(k)}\}$. Enforce

$$cm_{in}^{(k)} = cm_{k-1}, \quad cm_k = \text{SelectHead}(\{cm_{out}^{(k)}\}).$$

S3. Event semantics re-check (local witness only). Depending on etype_k , enforce the corresponding event constraints:

- **Produce:** constraints **P1–P9** instantiated with transcript x_k and witness w_k ;
- **Recycle:** constraints **R1–R8** instantiated with transcript x_k and witness w_k ;
- **Transfer (privacy-on):** constraints **T1–T4** instantiated with transcript x_k and witness w_k .

Crucially, w_k contains only information owned by the current event generator.

S4. State extraction. Enforce that $(st_k, \text{pid}_k, c_k, rc_k)$ equal the state fields of the selected head note at step k , as established by the event semantics constraints.

S5. Digest update. Update

$$D_k = H(D_{k-1} \parallel \ell_k),$$

binding the exact on-chain event sequence to the passport.

Binding the passport to the note (anti-substitution). To prevent a party from swapping in an unrelated passport artifact, we bind it to the produced head note. Concretely, the event generator includes a commitment to the passport state in the output note, e.g.,

$$\text{pc}_k := H(\text{tag}_{\text{pass}} \parallel D_k \parallel cm_k),$$

and enforces pc_k is absorbed into **attrs_com** (or another committed field) of the head output note. Thus, any later verifier can check that the presented passport corresponds to the on-chain lineage head.

Verification. An auditor verifies the final artifact $\mathcal{P}_T = (\Pi_T, D_T)$ by running $\text{IVC.Verify}(\Pi_T, S_T; \text{rt}_{\text{elog}})$. Verification cost is independent of the number of steps. No historical witnesses are revealed: witnesses are consumed only by the step prover that generated each event.

Roles. **Prover:** the supply-chain actor performing the current event (producer/recycler/transferer in privacy-on mode), who holds w_k . **Verifier:** any auditor/consumer/regulator who receives $\mathcal{P}_T$ from the current owner and checks Π_T . The ledger is *not* required to verify passport proofs; it continues to enforce per-event validity and freshness.

8 Embedding an LLM-based Proof Generator for Practical Deployment

8.1 Motivation

In real supply chains, many participants are non-experts with limited engineering resources. However, executing ZK-enabled events requires correct preparation of inputs (canonical encoding, units, evidence binding), selection of the appropriate policy/circuit template, and consistent generation/submission of transcripts and proofs. These operational requirements are a major barrier to deployment.

Our design targets a *fill-in* user experience: for a fixed industry and jurisdiction, processes, required certificates, and compliance predicates are largely standardized. Thus, users should only instantiate a pre-defined template by filling typed forms and attaching evidence, while the system deterministically assembles witnesses, generates proofs, and submits transactions.

8.2 System Components

C1. Template & policy registry $\mathcal{L}$ (versioned, curated). We maintain a versioned library $\mathcal{L}$ of reusable, audited artifacts:

- **Process templates:** event patterns and required fields per industry/jurisdiction.
- **Certificate schemas:** accepted credential types and canonical parsing rules; evidence-id derivation.
- **Policy & claim modules:** standardized predicates (threshold/range/grade/ratio) with jurisdiction parameters.

- **Circuit templates:** audited mappings from (etype, policy, modules) to circuits enforcing our event relations.

Each entry is uniquely identified and versioned, e.g.,

$$\text{tid} := H(\text{scope}||\text{name}||\text{version}),$$

and only identifiers in $\mathcal{L}$ are admissible for execution.

C2. Scope pinning (deterministic, non-LLM). A deterministic scope selector pins the applicable subset $\mathcal{L}[\text{scope}]$ based on user/enterprise metadata and UI choices (industry category, jurisdiction, product class). This prevents cross-domain drift (e.g., selecting the wrong national standard) and is enforced before any AI step.

C3. Agent orchestrator (LLM-based, untrusted). Within the pinned scope, an agent orchestrator produces a typed JSON intermediate representation (IR) that: (i) references only existing identifiers in $\mathcal{L}[\text{scope}]$, and (ii) fills only instance-level parameters from user-provided form entries. The agent never defines new schemas, constraints, or circuits. Its output is always treated as a draft.

C4. Deterministic validator & transpiler. A deterministic compiler validates the IR against the corresponding schema and policy rules, rejecting any: unknown identifiers, missing fields, out-of-range values, unit inconsistencies, or policy-incompatible event types. For a valid IR, it deterministically produces: (i) the exact public transcript x (matching our on-chain interface), (ii) a witness schema (canonical units/encodings and data mappings), (iii) the circuit template and proving configuration, and (iv) audit identifiers (template version, compiler version, circuit id). Thus, cryptographic semantics are fixed by deterministic compilation, not by the LLM.

C5. Local deterministic witness assembler (connectors). Sensitive witness data are never sent to the LLM. A local deterministic assembler populates witness values from enterprise sources (ERP/inventory, IoT logs, invoices/receipts, certificate documents) according to the witness schema. All normalization (units, fixed-point scaling, canonical serialization) is deterministic.

C6. Proving & submission service. The prover generates per-event SNARK proofs for ZK-enabled events. If passport is enabled, it additionally updates the carried passport artifact incrementally using only the current step’s witness. A relay fetches current public roots (e.g., rt_{cm} , rt_{elog}), prepares required membership paths, submits (etype, x , π), and monitors confirmation. Ledger-side freshness checks remain unchanged.

8.3 End-to-end Workflow for Non-expert Users

The user-facing workflow is deterministic and form-driven:

1. **Pin scope.** The user selects (or the system infers) industry/jurisdiction scope, fixing $\mathcal{L}[\text{scope}]$.
2. **Choose operation.** The user selects Produce, Recycle, or (optional) privacy-on Transfer.
3. **Fill typed form.** The UI renders a template-derived form: batch references (owned notes), quantities, output categories, and optional compliance claims. Fields are typed (enums, ranges, units) to prevent invalid entries.
4. **Attach evidence.** The user attaches required certificates/receipts. The system deterministically derives evidence identifiers (e.g., eid_{rec}) using canonical schema rules in $\mathcal{L}$.
5. **IR drafting (agent) → deterministic compilation.** The agent produces an IR draft; the validator/transpiler either rejects it with actionable diagnostics or compiles it into

$$(x, \text{witness-schema}, \text{circuit-config}).$$

6. **Local assembly & pre-check.** The local assembler populates the witness and runs a deterministic pre-checker mirroring the circuit constraints (e.g., conservation, yield bound, counter updates), producing human-readable errors if needed.

7. **Prove and (optional) passport update.** The prover generates π_{etype} . If enabled, it also updates the passport artifact $\mathcal{P}_{k-1} \mapsto \mathcal{P}_k$ using only current-step witness.
8. **Submit on-chain.** The relay submits the event transcript and proof. The ledger verifies π_{etype} when required, and enforces freshness (nullifiers/evidence ids).
9. **Deliver outputs.** The user receives updated note handles/paths and, if enabled, the updated passport artifact to be transferred alongside ownership.

8.4 Correctness, Safety, and Determinism

Closed-world execution. The agent can only reference identifiers present in $\mathcal{L}[\text{scope}]$; any invented schema/policy/claim/circuit is rejected. This prevents LLM hallucinations from affecting execution.

Deterministic Transpiler. All semantics (transcript layout, encodings, circuit selection) are fixed by the deterministic transpiler. Given the same validated IR and compiler/library versions, compilation yields identical outputs (auditable via template and circuit identifiers).

Witness isolation. All sensitive data remain local to the prover. The agent operates only on template-level descriptions and user-provided non-sensitive metadata. Privacy relies on ZK proofs, not on LLM behavior.

8.5 Template Management and Updates

Versioning and backward compatibility. Templates, policies, and schemas are versioned. Events record the policy/template identifier in their public transcript, ensuring verifiers can interpret constraints unambiguously. Older versions remain verifiable after updates.

Update workflow. New regulations or industry practices are incorporated by adding a new template/policy version to $\mathcal{L}$. Only audited entries are marked active for a given scope. Enterprises may pin to a specific version (stability) or opt-in to upgrades (compliance updates), enabling controlled rollouts.

8.6 Cost and Deployment Considerations

Lower engineering overhead (primary benefit). For SMEs, the dominant cost is integration and correctness rather than cryptography. Template-driven forms and deterministic compilation reduce misconfiguration and failed submissions, lowering operational support and deployment effort.

Template amortization. Once an industry/jurisdiction template is deployed, many participants reuse it without custom development, amortizing the cost of encoding certificate schemas, policies, and claim predicates.

Flexible proving modes. Participants can choose client-side proving, local server proving, or hosted proving depending on budgets. When hosted proving is used, confidentiality is preserved by keeping witnesses local (or using secure execution profiles), while the agent layer remains non-sensitive.

Passport overhead. The optional passport proof is maintained incrementally at each step, avoiding historical witness collection. Auditors verify a single final passport proof with constant verification cost; prover cost scales predictably with the number of steps.

Field / Symbol	Meaning & role	Realization (typ.)
st	Lifecycle state in $\{\text{raw}, \text{prod}, \text{waste}^{\text{in}}, \text{waste}^{\text{out}}, \text{spent}\}$; constrained by policy.	Small enum/int in witness; committed in cm .
pid	Production-line / epoch id; disambiguates line membership.	UUID.
c	Pipeline counter within a line; bounds/structures processing rounds.	Small int (optionally with range check, e.g., $0..K$).
rc	Recycle-count across lifecycles; increments only in Recycle .	Int; enforced $rc' := rc + 1$ in recycle circuit.
$owner$	Spending authority; enables transfer/authorization.	Pubkey/address; privacy-on: committed/hashed key; privacy-off: plaintext.
qty	Quantity (mass/volume); enables conservation & yield bounds.	Fixed-point int (e.g., grams with scale 10^d).
$attrs_com$	Commitment to sensitive attributes (supplier/recipe/measurements); binding without disclosure.	Hash/commitment over canonical encoding (Poseidon/Keccak).
$cred_com$	(Optional) commitment to external credential/certificate content.	$H(cred)$ or $H(issuer payload)$.
$policy$	Policy id selecting rules: Allowed , counter updates, ZK-on/off switches, claim types.	32-byte id (hash of policy spec/version); public input.
$link$	Traceability link binding outputs to consumed inputs (provenance without full disclosure).	$H(policy sorted(cm_{in}) salt)$ in circuit.
r	Commitment randomness for hiding.	Random nonce/scalar; witness only.
cm	Note commitment stored on-chain; leaf of NoteTree .	$cm = \text{Com}(\text{note}; r)$ (32 bytes).
nf	Nullifier for a spent note; prevents double-spend (freshness).	$nf = \text{PRF}(sk, cm)$; inserted into NullSet .
eid_{cert}	Admission evidence-id; prevents certificate replay.	$eid_{\text{cert}} = H(cred)$; insert into $\text{EidSet}^{\text{cert}}$.
eid_{rec}	Recycling evidence-id; prevents loop inflation.	$eid_{\text{rec}} = H(\text{rec.evidence})$; insert into $\text{EidSet}^{\text{rec}}$.
NoteTree / rt_{cm}	Public commitment structure anchoring membership of cm .	Incremental Merkle tree; root in contract state.
NullSet / rt_{nf}	Public set/accumulator of spent nf for freshness.	On-chain set (map) or accumulator; digest rt_{nf} .
EventLog	Append-only transcripts; auditable lifecycle trace.	Contract events; store hash+core fields; optional off-chain full data.

Table 1: Compact field dictionary. “privacy-on” hides suppliers/trading relations via ZK for selected events; “privacy-off” may reveal **owner** and spent references publicly.